

更新版本MAD

增加verdict状态，早退机制变成三层机制，summarize从答案一致变成立场一致。

需要重新做：

1. 三层summarize的输出json设计模板，每一层都需要记录reason才能判定立场一致
2. 三层早退机制不采用阈值三层机制，开放性任务数据集中效果不好再引入对应阈值
3. Debate Capsule合并到summary中，保留global votes、stable/unstable 标记、key claims

verdict 状态

include：所有 agent 都同意选这个选项（support_count = N）。已稳定，后续不再讨论。

exclude：所有 agent 都同意不选这个选项（support_count = 0）。已稳定，后续不再讨论。

disputed：既有 agent 支持也有 agent 反对（ $0 < \text{support_count} < N$ ）。未稳定，后续轮次需要继续讨论。

disputed 状态说明

向上连接 summarizer——压缩时 include 和 exclude 的选项各用一句话带过，disputed 的选项保留正反双方的完整理由。

向下连接早退——当所有选项的 verdict 都不是 disputed 时（全部为 include 或 exclude），等价于"全部稳定"，触发早退。只要还有一个 disputed 选项，辩论就应该继续。

向中间连接 prompt 构建——下一轮 agent 的 prompt 中只展开 disputed 选项的讨论，include 和 exclude 的选项只在 capsule 中用一句话提及。这直接减少了 prompt 长度。

完整的每轮流程总结

代码块

```
1 Round t 开始
2 |
3 |— Step 1: 链式辩论
4 |   Agent 1 发言 (读: Summary_{t-1} + Agent 3 的 R_{t-1})
5 |   Agent 2 发言 (读: Summary_{t-1} + Agent 1 的 R_t)
6 |   Agent 3 发言 (读: Summary_{t-1} + Agent 2 的 R_t)
7 |
8 |— Step 2: 统计 verdict
9 |   对每个选项 k, 统计 support_count_k
10 |   判定 verdict_k = include / exclude / disputed
11 |   计算 n_disputed
```

```

12 |
13 | — Step 3: 选择压缩级别
14 |   n_disputed = 0    → FULL (调用 summarizer)
15 |   n_disputed < K/2  → PARTIAL (调用 summarizer)
16 |   n_disputed ≥ K/2  → NONE (不调用 summarizer)
17 |   连续 K 轮 NONE    → 争议摘要 (调用 summarizer)
18 |
19 | — Step 4: 生成 Summary_t
20 |   FULL:    每个选项一句话理由
21 |   PARTIAL: 稳定选项一句话 + 争议选项保留正反理由
22 |   NONE:    只更新统计计数, 不生成摘要文本
23 |   争议摘要: 整理各方论据 + 找根源 + 给建议
24 |
25 | — Step 5: 早退判定
26 |   第一层: n_disputed = 0 (所有选项稳定) → 退出
27 |   第二层: 所有 verdict 连续 2 轮不变 → 退出
28 |   第三层: disputed 的 count 连续 2 轮不变 且 new_reasons = 0 → 退出
29 |   都不满足 → 进入 Round t+1
30 |
31 | — Step 6: 构建下一轮 prompt
32 |   include/exclude 选项: Summary 中的一句话
33 |   disputed 选项: Summary 中的正反理由 (PARTIAL) 或完整原文 (NONE)
34 |   前驱 agent 的原始回答: 直接传入

```

什么时候压缩

每轮结束后（所有 agent 都发完言后）压缩一次，不是每个 agent 发完言都压缩。

每轮结束后压缩一次的流程

```

1  Round t:
2    Agent 1 发言 (读取前驱 Agent 3 的 R_{t-1} 回答 + 上一轮 Summary_{t-1})
3    Agent 2 发言 (读取前驱 Agent 1 的 R_t 回答 + Summary_{t-1})
4    Agent 3 发言 (读取前驱 Agent 2 的 R_t 回答 + Summary_{t-1})
5
6    所有 agent 发完后:
7      统计每个选项的 verdict
8      选择压缩模板
9      调用 summarizer 生成 Summary_t
10     检查早退条件

```

每轮只调用 1 次 summarizer，Summary 基于本轮所有 3 个 agent 的最新回答生成，信息最完整。

三种压缩模板的完整设计

4.1 FULL 压缩模板

触发条件

所有选项的 verdict 都不是 disputed（全部为 include 或 exclude）。

Summarizer Prompt

代码块

```
1  你是辩论摘要 agent。本轮辩论结束后，所有 {N} 个 agent 对所有选项
2  的立场已完全一致。
3
4  各 agent 的回答：
5  {responses}
6
7  上一轮 Summary：
8  {prev_summary}
9
10  请完成以下任务：
11  1. 对每个选项，用一句话（≤30 tokens）说明被选入或排除的核心理由。
12  2. 如果某个选项在本轮刚从 disputed 变为 include/exclude，说明是什么
13     论据促成了这个变化。
14  3. 记录本轮和上一轮相比，哪些 agent 改变了哪些选项的立场。
15
16  输出严格 JSON，不要有任何其他内容。格式如下：
17  {
18    "round": <轮次>,
19    "compression_level": "full",
20    "option_ledger": {
21      "<选项>": {
22        "verdict": "include 或 exclude",
23        "reason": "一句话核心理由"
24      }
25    },
26    "final_answer": ["被选入的选项列表"],
27    "delta_from_last_round": {
28      "stance_changes": [
29        {"agent": <id>, "option": "<选项>", "from": "旧立场", "to": "新立场"}
30      ],
31      "options_newly_stabilized": ["本轮刚稳定的选项"],
32      "new_reasons": <本轮新增理由数>
33    }
34  }
```

输出示例

代码块

```

1  {
2    "round": 3,
3    "compression_level": "full",
4    "option_ledger": {
5      "A": {
6        "verdict": "include",
7        "reason": "所有 agent 一致认为 A 与题干中的关键症状描述直接匹配"
8      },
9      "B": {
10       "verdict": "exclude",
11       "reason": "B 所需的前提条件（肾功能异常）在题干中未体现"
12     },
13     "C": {
14       "verdict": "exclude",
15       "reason": "C 与题干中的年龄限制条件冲突"
16     },
17     "D": {
18       "verdict": "exclude",
19       "reason": "D 仅适用于急性期，题干描述的是慢性病程"
20     }
21   },
22   "final_answer": ["A"],
23   "delta_from_last_round": {
24     "stance_changes": [
25       {"agent": 2, "option": "B", "from": "support", "to": "oppose"}
26     ],
27     "options_newly_stabilized": ["B"],
28     "new_reasons": 0
29   }
30 }

```

这份 Summary 传入下一轮时的效果

原来 3 个 agent 各写了 400 token 的完整回答（共 1200 token）。FULL 压缩后变成约 200 token 的 Summary。下一轮每个 agent 的 prompt 中用 Summary 替代历史回答，节省约 1000 token。

4.2 PARTIAL 压缩模板

触发条件

有 disputed 选项但数量少于总选项数的一半。比如 4 个选项中 1 个 disputed。

Summarizer Prompt

代码块

```
1  你是辩论摘要 agent。本轮辩论结束后，部分选项已达成共识，部分仍有争议。
2
3  各 agent 的回答：
4  {responses}
5
6  上一轮 Summary：
7  {prev_summary}
8
9  当前各选项状态：
10 已稳定：{frozen_options 及其 verdict 列表}
11有争议：{disputed_options 列表}
12
13 请完成以下任务：
14 1. 对已稳定选项（include 或 exclude），各用一句话说明理由。
15 2. 对争议选项，分别列出支持方和反对方的 top-1 核心理由，
16    不要合并正方和反方的理由，它们必须分开保留。
17 3. 对争议选项，标注支持和反对的 agent 编号及数量。
18 4. 记录本轮立场变化。
19
20 输出严格 JSON，格式如下：
21 {
22   "round": <轮次>,
23   "compression_level": "partial",
24   "option_ledger": {
25     "<已稳定选项>": {
26       "verdict": "include 或 exclude",
27       "reason": "一句话核心理由"
28     },
29     "<争议选项>": {
30       "verdict": "disputed",
31       "support": {
32         "count": <支持数>,
33         "agents": [<agent 编号>],
34         "top_reason": "支持方核心理由"
35       },
36       "oppose": {
37         "count": <反对数>,
38         "agents": [<agent 编号>],
39         "top_reason": "反对方核心理由"
40       }
41     }
42   },
43   "frozen_options": ["已稳定选项列表"],
```

```

44     "disputed_options": ["争议选项列表"],
45     "current_answer": ["当前多数投票答案"],
46     "delta_from_last_round": {
47         "stance_changes": [...],
48         "options_newly_stabilized": [...],
49         "new_reasons": <数量>
50     }
51 }

```

输出示例

json

代码块

```

1  {
2      "round": 2,
3      "compression_level": "partial",
4      "option_ledger": {
5          "A": {
6              "verdict": "include",
7              "reason": "所有 agent 一致认为 A 与关键症状匹配"
8          },
9          "B": {
10             "verdict": "disputed",
11             "support": {
12                 "count": 1,
13                 "agents": [2],
14                 "top_reason": "B 的药理机制可解释题干中的治疗反应"
15             },
16             "oppose": {
17                 "count": 2,
18                 "agents": [1, 3],
19                 "top_reason": "B 缺少题干所需的核​​心适应症条件"
20             }
21         },
22         "C": {
23             "verdict": "exclude",
24             "reason": "所有 agent 同意 C 与年龄限制冲突"
25         },
26         "D": {
27             "verdict": "exclude",
28             "reason": "所有 agent 同意 D 仅适用于急性期"
29         }
30     },
31     "frozen_options": ["A", "C", "D"],
32     "disputed_options": ["B"],

```

```

33     "current_answer": ["A"],
34     "delta_from_last_round": {
35         "stance_changes": [
36             {"agent": 3, "option": "D", "from": "support", "to": "oppose"}
37         ],
38         "options_newly_stabilized": ["D"],
39         "new_reasons": 1
40     }
41 }

```

下一轮 agent 的 prompt 如何使用这份 Summary

代码块

```

1  [选项状态概览]
2  A: 已确认选入（所有 agent 一致）。理由：与关键症状匹配。
3  C: 已确认排除（所有 agent 一致）。理由：与年龄限制冲突。
4  D: 已确认排除（所有 agent 一致）。理由：仅适用于急性期。
5
6  [需要继续讨论的选项]
7  B: 有争议（1 票支持 vs 2 票反对）
8      支持方（Agent 2）：B 的药理机制可解释治疗反应
9      反对方（Agent 1, 3）：B 缺少核心适应症条件
10
11  请重点讨论选项 B，对已确认的 A/C/D 不需要再展开。

```

A、C、D 各一句话（约 60 token），B 展开了正反理由（约 80 token），总共约 140 token。比原始 $3 \times 400 = 1200$ token 节省了 88%，同时 B 的争议信息完整保留。

4.3 NONE 压缩模板

NONE 模式不调用 summarizer，但仍然需要更新选项级统计信息。输出的 JSON 是纯统计结果，不包含 summarizer 生成的理由摘要：

代码块

```

1  {
2      "round": 2,
3      "compression_level": "none",
4      "trigger": "n_disputed >= K/2",
5      "option_ledger": {
6          "A": {
7              "verdict": "disputed",
8              "support_count": 2,
9              "support_agents": [1, 3],
10             "oppose_count": 1,

```

```

11     "oppose_agents": [2]
12 },
13     "B": {
14         "verdict": "disputed",
15         "support_count": 1,
16         "support_agents": [2],
17         "oppose_count": 2,
18         "oppose_agents": [1, 3]
19     },
20     "C": {
21         "verdict": "exclude",
22         "support_count": 0,
23         "support_agents": [],
24         "oppose_count": 3,
25         "oppose_agents": [1, 2, 3]
26     },
27     "D": {
28         "verdict": "disputed",
29         "support_count": 1,
30         "support_agents": [3],
31         "oppose_count": 2,
32         "oppose_agents": [1, 2]
33     }
34 },
35     "frozen_options": ["C"],
36     "disputed_options": ["A", "B", "D"],
37     "current_answer_by_majority": ["A"],
38     "debate_capsule": "[Debate Status – Round 2]\n\nOption standings:\n  A: 2/3
include | verdict: disputed\n  B: 1/3 include | verdict: disputed\n  C: 0/3
include | verdict: exclude\n  D: 1/3 include | verdict: disputed\n\nStable
options:\n  C – excluded by all agents\n\nOptions under debate:\n  A (2 vs 1),
B (1 vs 2), D (1 vs 2)\n\nChanges this round:\n  Agent 3 switched on D:
support → oppose\n\nCurrent majority answer: [A]",
39     "delta_from_last_round": {
40         "stance_changes": [
41             {"agent": 3, "option": "D", "from": "support", "to": "oppose"}
42         ],
43         "new_reasons": 0,
44         "options_newly_stabilized": []
45     },
46     "consecutive_none_count": 2
47 }

```

NONE 模式结合 Capsule。之前 NONE 的 JSON 只有纯计数（谁支持谁反对），没有任何理由信息。agent 在下一轮要从前驱的完整原始回答（可能 400 token）中自己去理解“对方为什么这么选”——这对 LLM 来说是一个额外的认知负担，尤其当原始回答中包含大量无关的推理细节时。而 Capsule 恰好

解决了这个问题——它不做压缩（不丢信息），而是做**结构化整理**。把散落在各 agent 原始回答中的信息整理成一个清晰的"状态面板"，让下一轮 agent 快速了解全局态势。

早退机制

代码块

- 1 第一层早退：所有选项的 verdict 都不是 disputed → 全部稳定 → 退出
- 2 第二层早退：所有选项的 verdict 连续 2 轮不变 → 系统停滞 → 退出
- 3 第三层早退（deadlock）：disputed 选项的 support_count 连续 2 轮不变 且 无新理由 → 退出

对争议和平票的策略

对 disputed 选项的最终裁决

单选题： 最终只需要选一个答案。对所有 verdict=include 的选项取 support_count 最高的那个。如果没有 include 选项（极端情况），对所有 disputed 选项取 support_count 最高的。

代码块

- 1 如果有 include 选项：
- 2 final_answer = 唯一的 include 选项
- 3 如果没有 include 但有 disputed：
- 4 final_answer = disputed 选项中 support_count 最高的
- 5 如果平票（两个 disputed 选项 support_count 相同）：
- 6 final_answer = 最后一轮中最后发言的 agent 的选择

最后一条规则的理由：链式辩论中最后发言的 agent（Agent 3）读到了最多的信息（本轮 Agent 1 和 Agent 2 的最新回答），所以它的判断在信息量上是最充分的。这不需要额外调用任何 LLM，只是一个 tie-breaking 规则。

多选题： 对每个 disputed 选项独立裁决——support_count > N/2 就选入，≤ N/2 就不选。这就是 majority vote 在选项级别的应用。

代码块

- 1 对每个 disputed 选项 k：
- 2 如果 support_count_k > N/2 → 选入
- 3 如果 support_count_k < N/2 → 不选
- 4 如果 support_count_k = N/2（精确平票，仅 N 为偶数时可能）：
- 5 参考最后发言 agent 的立场

为什么不用 confidence-weighted vote

PDF 中建议对争议选项用 confidence-weighted vote。但 iMAD 和 ConfMAD 都证明了 LLM 置信度不可靠。如果在最终聚合阶段引入 confidence，相当于在最关键的决策点上依赖了最不可靠的信号。support_count（纯计数）虽然信息量少，但完全客观，不会因为某个模型"过度自信"而被带偏。

实验设置

模型设置：三种异构轻量模型

选择原则：三个模型来自不同公司/架构系列，参数量都在 7-8B 级别（单卡 24GB 可跑），均有 Instruct 版本（适合辩论场景），且在推理 benchmark 上各有优劣势（提供观点多样性）。

模型	公司	参数量	优势领域	劣势领域
Qwen2.5-7B-Instruct	阿里	7.6B	数学推理、多语言、代码	英文常识略弱于 LLaMA
Llama-3.1-8B-Instruct	Meta	8B	英文理解、指令跟随、长文本 (128K)	数学推理不如 Qwen
Mistral-7B-Instruct-v0.3	Mistral AI	7.2B	推理速度快、推理效率高	上下文窗口短(32K)、

数据集设置

50 题来自 MMLU-Pro test split，覆盖 13 个学科类别，分布为：physics(8)、chemistry(7)、law(7)、business(5)、engineering(5)、economics(4)、psychology(3)、other(3)、math(2)、health(2)、history(2)、biology(1)、philosophy(1)。